Kara Muhafız — Gün 1-2 Çalışma Raporu

ROS 2 Humble + Gazebo Classic — Ortam Kurulumu ve Simülasyon Ayağa Kaldırma (2026-10-03)

---

1. Ortam Kontrolü (Gün 1)

Mevcut kurulum doğrulandı, eksik bir şey çıkmadı:

- Ubuntu 22.04.5 LTS (Jammy)
- ROS 2 Humble Hawksbill
- Gazebo Classic 11.10.2
- gazebo_ros2_control, ros2_control, ros2_controllers, gazebo_ros
- Nav2 paket ailesi (amcl, bt_navigator, bringup, ...)
- robot_localization
- depthimage_to_laserscan, teleop_twist_keyboard, joint_state_publisher
- colcon, xacro
- Eksik: twist_mux (3. haftada kurulacak)

2. Workspace ve Paketleme (Gün 2)

Proje kökünde src/ klasörü oluşturulup verilen karamuhafiz_base klasörü içine taşındı. Şartname gereği içerik (urdf/mesh/world) hiç değiştirilmedi — yalnızca ROS 2'nin paket olarak tanıyabilmesi için package.xml ve CMakeLists.txt eklendi.

Oluşturulan/değişen dosyalar:

- src/karamuhafiz_base/package.xml, CMakeLists.txt (yeni — paket tanımı)
- src/karamuhafiz_bringup/package.xml, CMakeLists.txt (yeni paket)
- src/karamuhafiz_bringup/config/controllers.yaml (yeni — şimdilik yalnızca joint_state_broadcaster)
- src/karamuhafiz_bringup/launch/sim_bringup.launch.py (yeni — Gazebo + parkur + araç + robot_state_publisher + spawn + joint_state_broadcaster)

karamuhafiz.urdf.xacro, teknofest_parkur.world ve mesh dosyalarının içeriği hiç değiştirilmedi.

3. Ulaşılan Sonuç

- Gazebo açılıyor, parkur (bariyer, koni, tabela, rampa) ve araç doğru yükleniyor.
- robot_state_publisher TF zincirini yayınlıyor.
- controller_manager başarıyla ayağa kalkıyor, joint_state_broadcaster ~150 Hz ile /joint_states yayını yapıyor (ros2 topic hz ile doğrulandı).

4. Karşılaşılan Hatalar ve Çözümleri

4.1 rosdep bulunamadı

Belirti: "Command 'rosdep' not found". Çözüm: Zararsız — colcon build zaten başarıyla tamamlandı, rosdep olmadan devam edildi.

4.2 Gazebo ikinci açılışta boş geldi

Belirti: Model listesi boş, sim time/iterations donmuş. Sebep: Ctrl+C sonrası eski gzserver/gzclient süreçleri tam kapanmadan yeni launch üstüne denk geldi. Çözüm: Her yeniden başlatmadan önce pkill -9 -f gzserver / gzclient ile temiz kapatma alışkanlığı edinildi.

4.3 Ana sorun: controller_manager hiç başlamıyordu (joint_state_broadcaster sonsuz bekliyordu)

Belirti: ros2 topic info /joint_states → Publisher count: 0. spawner_joint_state_broadcaster sürekli "waiting for service /controller_manager/list_controllers" diyordu.

Teşhis süreci: Launch logları incelendi, gzserver loglarında şu hata görüldü:

```
[gazebo_ros2_control]: parser error Couldn't parse parameter override rule:
'--param robot_description:=<?xml version="1.0" ?>...'
```

Kök sebep: karamuhafiz.urdf.xacro içindeki gazebo_ros2_control plugin bloğu <robot_param_node>robot_state_publisher</robot_param_node> kullanıyor — bu, eklentinin robot_description'ı robot_state_publisher node'undan uzaktan çekip controller_manager node'una "--param robot_description:=<ham XML>" şeklinde bir komut satırı override'ı olarak aktarmasına sebep oluyor. Bu değer düşük seviyede (rcl/arguments.c) YAML olarak ayrıştırılıyor; XML yorumlarındaki (Türkçe açıklamalar) "kelime: kelime" (iki nokta üst üste + boşluk) kalıpları YAML'e yeni bir anahtar-değer çiftinin başladığı izlenimini verip ayrıştırmayı bozuyor. Bu, ros-humble-gazebo-ros2-control 0.4.10 paketinin bilinen bir hatası; apt güncellemesi düzeltmedi (Humble bu sürüm hattında donmuş).

Denenen ama işe yaramayan yollar: SDF'e önceden çevirip dosyadan spawn etmek (bug farklı bir kod yolunda), robot_description'ı tek satıra sıkıştırmak (yorumlar kaldırılmadan yetersiz kaldı), robot_state_publisher node adını değiştirmek (eklentinin bu isimde sabit bir servis beklediği, fallback olmadığı ortaya çıktı).

Uygulanan çözüm: sim_bringup.launch.py içinde, robot_state_publisher'a verilen robot_description parametresi, xacro çıktısından yalnızca bellekte (regex ile) XML yorumları temizlenip tek satıra sıkıştırılarak üretiliyor. Asıl .xacro dosyasının içeriği hiç değiştirilmedi — yalnızca canlı parametreye giden kopya işlendi. Bu, şartnamenin "verilen dosyalar değiştirilemez" kuralına tam uyumlu.

Doğrulama: ros2 topic hz /joint_states → ortalama 150 Hz, controller_manager servisleri aktif.

5. Kod Analizi — Oluşturulan Dosyaların Satır Satır İncelenmesi

Bu bölüm, bugün yazılan her dosyanın önemli kısımlarını açıklar — final sözlü değerlendirmede bu kodu satır satır savunabilmen için.

5.1 src/karamuhafiz_base/package.xml

<exec_depend>xacro</exec_depend> vb. satırlar, bu paketin ÇALIŞMA ZAMANINDA (derleme zamanında değil) hangi paketlere ihtiyaç duyduğunu bildirir. colcon, bağımlılık grafiğini buradan çıkarıp paketleri doğru sırada derler. <buildtool_depend>ament_cmake</buildtool_depend> ise bu paketin CMake tabanlı (ament_cmake) bir ROS 2 paketi olduğunu belirtir — Python paketleri için bunun yerine ament_python kullanılır.

5.2 src/karamuhafiz_base/CMakeLists.txt

```
install(
  DIRECTORY urdf meshes worlds
  DESTINATION share/${PROJECT_NAME}
)
```

Bu üç satır en kritik kısım: src/ içindeki urdf, meshes, worlds klasörlerini colcon build sırasında install/karamuhafiz_base/share/karamuhafiz_base/ altına kopyalar. get_package_share_directory("karamuhafiz_base") gibi Python fonksiyonları (launch dosyalarında kullandık) dosyaları işte bu standart "share" konumundan bulur. Bu satır olmasaydı, launch dosyası xacro/world dosyalarını asla bulamazdı. ament_package() ise paketin ament index'e (ROS 2'nin paket kayıt sistemi) kaydedilmesini sağlayan zorunlu son çağrı.

5.3 src/karamuhafiz_bringup/config/controllers.yaml

```
controller_manager:
  ros__parameters:
    update_rate: 100
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster
```

ros__parameters anahtarı, ROS 2'nin YAML parametre dosyalarında kullandığı standart bir isimlendirme (node adının altında yer alır — burada node adı "controller_manager"). update_rate, controller_manager'ın saniyede kaç kez tüm controller'ları güncelleyeceğini (100 Hz) belirler — bu,

fiziksel kontrol döngüsünün hızıdır. joint_state_broadcaster alt anahtarı, controller_manager'a "bu isimde, bu TİPTE bir controller yükleyebilirsin" der; gerçek yükleme işini spawner node'u (launch dosyasında controller_manager/spawner) yapar.

5.4 src/karamuhafiz_bringup/launch/sim_bringup.launch.py

```
urdf_xml = subprocess.run(
    ["xacro", xacro_file, f"controllers_yaml:={controllers_yaml}"],
    check=True, capture_output=True, text=True,
).stdout
```

xacro aracını bir alt süreç olarak çalıştırıp .xacro dosyasını gerçek URDF/XML metnine çeviriyoruz. controllers_yaml:=... kısmı, xacro dosyasındaki <xacro:arg name="controllers_yaml"/> argümanına bizim ürettiğimiz controllers.yaml'ın tam dosya yolunu geçiriyor — bu yol, xacro içindeki $(arg controllers_yaml) ifadesinin yerine geçecek.

```
urdf_no_comments = re.sub(r"<!--.*?-->", "", urdf_xml, flags=re.DOTALL)
urdf_xml_flat = " ".join(urdf_no_comments.split())
robot_description = ParameterValue(urdf_xml_flat, value_type=str)
```

Bugünkü ana hata düzeltmesi (bkz. Bölüm 4.3): regex ile XML yorumları (<!-- ... -->) temizleniyor, sonra tüm metin boşluklara göre parçalanıp tek boşlukla yeniden birleştiriliyor (tüm satır sonları/fazla boşluklar kayboluyor, XML geçerliliği bozulmuyor çünkü XML'de boşluk anlamsızdır). ParameterValue(..., value_type=str) ise ROS 2'ye bu parametrenin KESİNLİKLE bir string olduğunu açıkça söyler — aksi halde ROS 2 içeriğe bakıp yanlış bir tip (örn. sayı) çıkarmaya çalışabilir.

```
gzserver = ExecuteProcess(
    cmd=["gzserver", world_file,
         "-s", "libgazebo_ros_init.so",
         "-s", "libgazebo_ros_factory.so"],
    output="screen",
)
```

gzserver (Gazebo'nun fizik motorunu/arka ucunu çalıştıran süreç), ExecuteProcess ile düz bir komut satırı programı gibi başlatılıyor (bu bir ROS 2 node'u değil, bu yüzden Node() değil ExecuteProcess() kullanıyoruz). -s bayrakları iki Gazebo eklentisini (sistem seviyesinde) yüklüyor: libgazebo_ros_init.so saat (/clock) ve temel ROS köprüsünü, libgazebo_ros_factory.so ise /spawn_entity servisini (modelleri çalışan simülasyona sonradan ekleyebilme yeteneğini) sağlıyor — spawn_entity.py'nin çalışabilmesi tamamen bu ikinci eklentiye bağlı.

```
robot_state_publisher = Node(
    package="robot_state_publisher",
    executable="robot_state_publisher",
    output="screen",
    parameters=[{"use_sim_time": True, "robot_description": robot_description}],
)
```

Node(...) bu sefer gerçek bir ROS 2 node'u başlatıyor. use_sim_time: True çok önemli — bu olmazsa node, gerçek duvar saatini kullanır ama Gazebo kendi simülasyon saatini (/clock) yayınlar; ikisi senkronize olmazsa TF zaman damgaları tutarsız olur ve RViz'de "TF_OLD_DATA" gibi hatalar görülür. robot_description parametresi, bu node'un TF ağacını ve /robot_description topic'ini üretmek için okuduğu URDF metni — gazebo_ros2_control eklentisinin (robot_param_node mekanizmasıyla) tam olarak buradan çektiği değer de bu.

```
spawn_entity = Node(
    package="gazebo_ros",
    executable="spawn_entity.py",
    arguments=["-file", sdf_path, "-entity", "karamuhafiz", "-z", "0.05"],
)
```

-file ile önceden gz sdf -p komutuyla ürettiğimiz statik SDF dosyasından spawn ediyoruz (canlı /robot_description topic'inden değil — bu, erken denemelerde denediğimiz ama farklı bir bug'a takılan bir yoldu, şu an asıl çalışmayı sağlayan robot_description düzeltmesi zaten ayrı). -entity karamuhafiz, Gazebo içindeki modelin adı; -z 0.05 modeli zeminin 5 cm üzerinde doğurarak başlangıç çarpışmasını önlüyor.

6. Bugün Çalışılması Gereken ROS 2 / Robotik Kavramları

- ROS 2 workspace yapısı: src/ → colcon build → install/ (ve --symlink-install farkı)
- package.xml ve CMakeLists.txt'nin görevi; ament_cmake install() mekanizması
- xacro:arg ve $(arg ...) ile değer geçişi (ROS1 roslaunch söz dizimiyle aynı köken)
- Launch dosyası (Python launch): Node, ExecuteProcess, substitution (Command) kavramı
- robot_state_publisher: URDF'den TF zincirini nasıl üretir
- ros2_control mimarisi: controller_manager, hardware interface (GazeboSystem), controller spawner (joint_state_broadcaster)
- ROS 2 parametre override mekanizması (--param, --params-file) ve bunun YAML tabanlı ayrıştırılması — neden içerik hassasiyeti olabileceği
- Debugging yöntemi: log dosyasına yönlendirme (tee), grep ile hata izleme, ros2 topic info/hz ile canlı teşhis

7. Sıradaki Adım (Gün 3)

controllers.yaml içine direksiyon (steering, position komutlu) ve tahrik (traction, velocity komutlu) controller'larının eklenmesi; Ackermann sürüş mantığının kurulması; TF ağacının RViz2'de tam doğrulanması.

Bu rapor, Kara Muhafiz ROS 2 2025 Parkur Görevi 4 haftalık yol haritasının (Kara_Muhafiz_ROS2_Yol_Haritasi.pdf) Hafta 1 / Gün 1-2 kapsamını özetler.